

Rapport d'avancement (3ème semaine) — SAE 5.01

OUKARA Anass

KIRCHER Nino

SCHOU Lilian

PLANCHER Mattéo

1. Base de données

La base de données est désormais opérationnelle. Cette dernière est également connectée à l'application via un fichier de connexion.

Étapes de connexion (pour macOS, mais similaire sur Linux et Windows) :

- Installer Postgres.app.
- Cliquer sur le + en bas à gauche pour créer un nouveau serveur appelé « Manganitor ».
- Stocker ce serveur dans un dossier différent de celui proposé par défaut (car un autre serveur est déjà présent dedans).
- Cliquer sur « Initialize ».
- Cliquer sur « Connect », puis dans la fenêtre pop-up qui s'ouvre, mettre ces informations :
 - Database : Postgres
 - User : *nom d'utilisateur*
 - Client : Terminal
- Dans le terminal ouvert automatiquement :
 - Taper **CREATE DATABASE manganitor;** pour créer une nouvelle base de données appelée « manganitor ».
 - Taper **\c manganitor** pour accéder à cette base.
 - Ne pas copier et coller le script SQL. À la place le stocker dans un dossier puis exécuter la commande :

\i 'chemin_du_fichier/init_db.sql'

- Taper **\dt** pour s'assurer que les 2 tables ont bien été créées.

2. Utilisation de l'IA (Lilian)

Utilisations de l'IA :

- Comparaison des différentes méthodes de hachage de mots de passe (SHA-256, MD5...) avec Gemini Pro. Cela a permis de découvrir de nouvelles méthodes de hachage telles qu'Argon2id ou bcrypt.

Non-utilisation de l'IA :

- Architecture de la base de données : tables, attributs, relations, MCD (au brouillon).

Utilisation en mode hybride :

- Script SQL de création de la base de données : un premier code SQL a été réalisé intégralement à la main à partir du recueil des besoins et de l'analyse du sujet. Plusieurs tests et vérifications ont ensuite été effectués, puis le code a été donné à une IA (ChatGPT) dans le but de vérifier les éventuelles erreurs et d'obtenir quelques pistes d'amélioration possibles.

Ces suggestions ont été ignorées (stockage de la version du modèle de détection des mangas, informer si une image a été ajoutée depuis la galerie de l'utilisateur ou prise directement avec l'appareil photo...) car leurs apports ont été jugés inutiles.

3. Backend : sécurisation, PostgreSQL et contributions

- Sécuriser les identifiants utilisateurs en UUID.
- Les contributions des utilisateurs sont maintenant enregistrées dans l'historique.
- Réorganiser le code du serveur pour que chaque fonctionnalité soit mieux séparée.
- Ajouter une protection par identifiants et mot de passe/clé pour sécuriser les actions sensibles.
- Changé de système de base de données pour un plus robuste (PostgreSQL).
- Créer une page dédiée pour l'envoi des contributions par les utilisateurs.

4. Déploiement et améliorations

- Déploiement de notre API et DB sur docker sur un serveur personnelle
- Intégration de l'API sur notre front-end
- Ajout de l'historique
- Sauvegarde cache pour les requêtes
- Amélioration de l'intégration de manga-dex
- Refactoring de code